Today
Welcome to your next lesson: Making code speak for itself.

I'm getting your personalized lesson ready — based on what you've learned, what's next, and the best way to lock in your skills.

Step by step, you’re

moving forward.

Let’s start with one simple idea today: good names make code easy to read 🧠✨

Imagine you see this in someone else’s Python file:

Python

```
def f(x, y):
```

```
    return x * 9/5 + 32
```

No comments, no context.

f – what does it do?
x, y – what do they mean?
You have to think hard just to guess the purpose.

Now compare:

```
Python

def
```

```
celsius_to_fahrenheit(cel
sius):
    return celsius * 9/5
+ 32
```

You almost don’t need an explanation — the name explains the intent.

Quick check-in: if you had to choose between these two function names for that conversion, which feels clearer to future you when you come back in 3 months?

f
celsius_to_fahrenheit

Just answer with the name you'd pick, and then we'll talk about Python's naming style step by step.

celsius_to_fahrenheit

Exactly 🎯

You just felt the core idea of this lesson: names should explain what the code does so you don’t have to “decode” it later.

Now, a key Python style rule: we usually write function names in snake_case – all lowercase, words separated by underscores,

like:

celsius_to_fahrenheit
apply_tax
calculate_total_price

This is preferred over things like celsiusToFahrenheit or calcTemp.

Quick question for you: have you seen snake_case vs camelCase before, or

should I show you a super short side-by-side example next?

it was introduced in python 1010yes

Nice, so this will be a quick level-up, not brand new stuff 💪

Let’s zoom in on function naming in Python, one step at a time.

In Python, good function names are usually:

snake_case (lowercase, words with _)
verbs or verb phrases (they do something)
clear about intent, not cryptic

Example:

Python

```
def t(a, b):
    return a + b
```

vs

Python

```
def add_tax(subtotal,
tax_rate):
    return subtotal +
subtotal * tax_rate
```

Second one tells you:

It adds tax
It uses a subtotal
It uses a tax_rate
Before I quiz you with a tiny multiple-choice question, want a quick visual table showing good vs weak function names, or do you feel ready to be quizzed right away?

I'm ready for the quiz

What do you think? Pick the one that looks most

like normal Python 🐍 style.

Which function name best follows Python’s usual style for a function that calculates a user’s final bill total?

A.

billTotal

B.

CalculateFinalBill

C.

calc_final_bill

D.

finalBillCalc

Nice pick, Ron 🎯 calc_final_bill is exactly what we want here.

It uses snake_case → all lowercase with _ between words
It starts with a verb (calc = calculate) → functions do things
It clearly hints at the purpose: calculate the final bill

The others either use camelCase (billTotal, finalBillCalc) or start with a capital letter

like a class name (CalculateFinalBill), which goes against usual Python style for functions.

Next tiny step: let’s look at how a good name plus a short docstring can make a function almost self-explanatory. Ready for that?

Yes, I'm ready

Awesome 👏

Next concept: docstrings – a built-in way to describe what a function does.

Look at this:

Python

```
def
celsius_to_fahrenheit(cel
```

```
sius):
    """Convert a
temperature from Celsius
to Fahrenheit."""
    return celsius * 9/5
+ 32
```

That triple-quoted string right under the def line is the docstring.

Plain English, one short sentence: what this function does.

So structure is:

Python

```
def function_name(...):
    """Short description
of what the function
does."""
    # code here
```

Does that idea make sense so far: docstring = short description right under the function definition?

yes it makese sense and

ive sen -> Integer after parameter to descrivebe the type its gong t o return

Nice, you’re connecting things well 👌

What you’ve seen before is probably type hints like:

Python

```
def add(a: int, b: int)
-> int:
    """Add two integers
and return the result."""
    return a + b
```

a: int and b: int describe the parameter types

-> int describes the return type

The docstring describes the purpose in words

For this lesson, we'll keep type hints very

light and focus on clear names + clear docstrings.

Next small step: I'll show you one messy function and one cleaned-up version, then I'll ask you to rename another one yourself. Sound good?

quick question whas the though behind not just using #This is what my funcion does which also

works

Love that question 🔥

Short answer: # This is what my function does and a docstring are not the same thing.

Here’s the key difference:

Python

```
def
celsius_to_fahrenheit(cel
sius):
    """Convert a
temperature from Celsius
to Fahrenheit."""
    return celsius * 9/5
+ 32
```

vs

Python

```
def
celsius_to_fahrenheit(cel
```

```
sius):
    # Convert a
temperature from Celsius
to Fahrenheit.
    return celsius * 9/5
+ 32
```

Step by step:

Docstrings live inside the function, as the first line, and Python can read them at runtime with help() or tools/IDEs.

Comments (# ...) are only for humans reading the file; Python ignores them completely and they’re not accessible via help().

So:

Use docstrings to describe what the function is for.

Use comments to explain tricky parts of the logic.

Does that distinction between “function description = docstring” and “explain tricky line = comment” feel clear?

now it makes a lot more sense parsable tag as it were

Exactly — I like how you put that: it’s like a parsable tag Python and tools can read 💡

Quick visual:

Python

```
def apply_discount(price, discount):
    """Apply a discount to a price and return the final amount."""
```

```
    # Make sure discount
is between 0 and 1
    final_price = price *
(1 - discount)
    return final_price
```

Docstring → high-level “what this function is for”

Comment → why this line or step matters / is non-obvious

Next tiny step: I’ll show you a messy function and a cleaned-up version (better name +

docstring), then you'll try renaming one yourself. Ready for that?

Yes, I'm ready

Nice 😄 let's do this step by step.

Here's a messy but working function:

Python

```
def f(a, b):
    r = a * 1.0825
    return r
```

You can probably guess it's adding some kind of tax or fee, but:

f doesn't say what it does

a, b, r don't say what they mean

There's no docstring

A clearer version might

look like:

Python

```
def
apply_sales_tax(subtotal)
:
    """Calculate total
price after applying
sales tax."""
    total_with_tax =
subtotal * 1.0825
    return total_with_tax
```

Now the name + docstring almost read like English.

Your turn (just thinking, no code yet):

If you saw `apply_sales_tax(subtotal)` in a file you'd never seen before, would you feel like you know what it does without reading the body?

Yes, it's much clearer

Perfect — that's exactly

the instinct we want to build 🧠✨

Next mini-skill: writing a short docstring for a well-named function.

Pattern I want you to use:

One short sentence,

starting with a verb, saying what the function does.

Example:

Python

```
def apply_sales_tax(subtotal):
    """Calculate total price after applying
```

```
sales tax."""
    total_with_tax =
subtotal * 1.0825
    return total_with_tax
Docstring checklist:


Inside triple quotes """
... """
On the very first line
inside the function
One clear English
sentence
Want to try a tiny quiz
on where the docstring
```

should go and what it should look like?

Yes, let's try the quiz

What do you think? Pick the option that matches how Python expects docstrings to be written 👇

Where should a function’s docstring go in Python?

A.

Right above the function, starting with #

B.

As the first line inside the function, in triple quotes """

C.

At the very end of the file, in triple quotes

D.

Anywhere inside the function, as long as it’s a # comment

Nice work, Ron 🎉 You nailed it.

A function docstring should be:

Python

```
def some_function():
    """Short description
of what the function
does."""
    # rest of the code
First line inside the
function body
In triple quotes """ ...
""" so Python and tools
can read it with help()

# comments are just for
humans reading the file
and aren’t treated as
docstrings.
```

Next step: let’s have you rename a messy function and add a docstring yourself. I’ll describe it, and you’ll do the cleanup in the editor. Ready for a small hands-on refactor?

I'm ready!

Great 🔥 Let’s do a small refactor together.

Step 1 – I’ll give you a messy function to clean up.

In your code editor, type this version first:

Python

```
def g(x, y):
    z = x / y
    return z
```

Don’t change it yet—just

get this into the editor and Run it once to be sure it works (you can pick any numbers for a quick test).

Then hit Submit code so I can see it.

After that, I’ll ask you to:

rename g, x, y, z to
clear names, and
add a one-sentence
docstring.

Go ahead and submit this
starting version, and
tell me when it’s
submitted.

Python

```
def g(x, y):
    z = x / y
    return z
```

Nice, you got the starting version in 👍

Now let’s clean it up step by step.

This function just divides one number by another, so we want:

A clear function name (verb phrase)

Clear parameter names
A short docstring describing what it does

Suggested changes:

Rename g → safe_divide or divide_numbers
Rename x → numerator (or dividend)
Rename y → denominator (or divisor)
Rename z → result
Add a docstring like:

"""Divide one number by another and return the result."""

So the structure should look like:

Python

```
def better_name(better_param1, better_param2):
    """Short description of what the function does."""
```

```
    better_variable = ...
    return
better_variable
```

Your turn 💪

In the code editor, update your function:

pick good names (your own choices are fine if they’re clear and snake_case)
add a one-sentence

docstring in triple quotes as the first line inside

Then Run to check it, and Submit code so I can review.

Python

```
def divide_numbers(numerator, denominator):
    """Divide fist number by the seond number and
```

```
return the result."""
    result = numerator /
denominator
    return result
print(f"5 / 2 =
{divide_numbers(5,2)}")
```

Ron, this is excellent 👏

Let’s check it against our goals:

Python

def

```
divide_numbers(numerator,
denominator):
    """Divide fist number
by the seond number and
return the result."""
    result = numerator /
denominator
    return result
```

Step by step:

divide_numbers → clear verb phrase, snake_case ☑

numerator, denominator →

very clear parameter names ☑
result → clear variable name ☑
Docstring → first line inside the function, in triple quotes ☑
Test call with print(...) → great way to confirm behavior ☑

Only tiny tweak I’d suggest is just spelling in the docstring (not about Python, just

clarity):

Python

"""Divide the first
number by the second
number and return the
result."""
But structurally, you did
exactly what a pro would
do here 👌

Next micro-skill for this
lesson: using inline
comments (# ...) only

when a line is non-obvious, not to restate what the name or docstring already say.

Do you want to see a quick example of “too many comments” vs “just enough comments,” or do you feel clear on the difference already?

sorry got a crappy key with some sticky keys

makes me sloppy

All good, no worries at all 😄 — you’re doing solid work.

The nice thing is: Python doesn’t care about spelling in docstrings, and your code itself was clean and well-named, which is what matters most here.

Since you've already nailed:

good function naming (divide_numbers)
clear parameters (numerator, denominator)
a proper docstring in the right place

Next tiny step is: when to add an inline # comment and when to skip it.

Want a quick side-by-side example of:

“too many / useless comments” vs
“just enough comments for the tricky part”?
Yes, please show an example

Nice, let’s keep this tight and visual 👀

Here’s too many / useless comments:

Python

```
def
divide_numbers(numerator,
denominator):
    """Divide the first
number by the second
number and return the
result."""
    # set result equal to
```

```
numerator divided by
denominator
    result = numerator /
denominator
    # return result
    return result
```

Those comments don’t add anything – they just repeat what the code already says.

Here’s just enough commenting on a non-obvious step:

```
Python

def
apply_step_discount(price
, step):
    """Apply a discount
that gets bigger in fixed
steps."""
    # Round down to
nearest 'step' before
applying discount logic
    stepped_price =
(price // step) * step
    return stepped_price
The name + docstring
```

explain what the function is for
The inline comment explains why that line matters (not obvious from code alone)
Does this rule feel clear?

Comment why when the reason isn’t obvious.

Don’t comment what if the

name and code already say it.

If that makes sense, I'll give you one small multiple-choice question to lock it in.

It makes sense!

What do you think? Pick the version that best follows how we really

want to use comments in Python 👇

Which version uses comments in the MOST helpful Python style?

A.

```
def multiply(a, b):
    """Multiply two
numbers and return the
result."""
    # multiply a and b
    result = a * b
```

```
    # return the result
    return result
```

B.

```
def multiply(a, b):
    """Multiply two
numbers and return the
result."""
    result = a * b
    return result
```

C.

```
def multiply(a, b):
    # This function
multiplies a and b and
```

returns the result
    result = a * b
    return result
D.

def multiply(a, b):
    """Multiply two
numbers and return the
result."""
    # Use multiplication
here instead of addition
    result = a * b
    return result
Nice choice, Ron 🎯

This version:

Python

```
def multiply(a, b):
    """Multiply two
numbers and return the
result."""
    result = a * b
    return result
```

is best because:

The docstring clearly

explains what the function does ☑
The code is already obvious, so no extra comments are needed ☑
We’re not repeating the same sentence as a # comment or adding “captain obvious” comments

You’re using docstrings and comments exactly how a professional Python dev would 👌

You’ve now shown you can:

pick good, Pythonic function names
write clear docstrings in the right place
use comments only when they really add value

That’s the full skill set for this lesson ☑

Want to wrap up this lesson and move on to the next one, or do you want one more small code refactor practice just for extra confidence?

Let's move on

Awesome work today, Ron 🎉

In this lesson you:

Used clear, Pythonic

function names (divide_numbers, numerator, denominator) so the code explains itself.
Wrote proper docstrings as the first line inside functions to describe what they do.
Learned to use inline comments only when they explain why something non-obvious is happening, not to restate obvious code.

You’re writing code that your future self (and teammates) will actually enjoy reading — that’s a big professional step 💪

When you’re ready, you can jump into the next lesson from the Learn tab, and I’ll be right here to go through it with you.